

Cetus DLMM

Security Assessment

November 4th, 2025 — Prepared by OtterSec

Michał Bochnak embe221ed@osec.io

Thiago Tavares thitav@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-CDM-ADV-00 | Absence of Verification for Reward Start Timestamp 6

OS-CDM-ADV-01 | Improper Rounding 7

General Findings **8**

OS-CDM-SUG-00 | Invalid Recipient Address 9

OS-CDM-SUG-01 | Risk of Overflow in Fee/Reward Calculation 10

OS-CDM-SUG-02 | Code Refactoring 11

OS-CDM-SUG-03 | Code Maturity 12

Appendices

Vulnerability Rating Scale **14**

Procedure **15**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Cetus Protocol engaged OtterSec to assess the `cetus-dlmm`, `integer-mate`, and `move-stl` programs. This assessment was conducted between September 17th and October 21st, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 6 findings throughout this audit engagement.

In particular, we identified a vulnerability that allows rewards to start before the expected timestamp, breaking the reward settlement calculations, resulting in pool operations failing and creating a denial-of-service scenario (OS-CDM-ADV-00). Additionally, due to the current rounding logic, small deposits will yield zero shares, resulting in users losing tokens without receiving any corresponding liquidity shares (OS-CDM-ADV-01).

We also made suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices (OS-CDM-SUG-03), and recommended modifying the codebase to improve consistency, functionality, and removal of unnecessary code (OS-CDM-SUG-02). We further advised to add explicit assertions for overflows or utilize saturating addition when increasing the fee/reward amounts (OS-CDM-SUG-01).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in Git repositories at https://github.com/CetusProtocol/cetus-dlmm, github.com/CetusProtocol/integer-mate, and github.com/CetusProtocol/move-stl. This audit was performed against cd9ab3f (for cetus-dlmm), 1fc772f (for integer-mate), and 0681fec (for move-stl).

A brief description of the program is as follows:

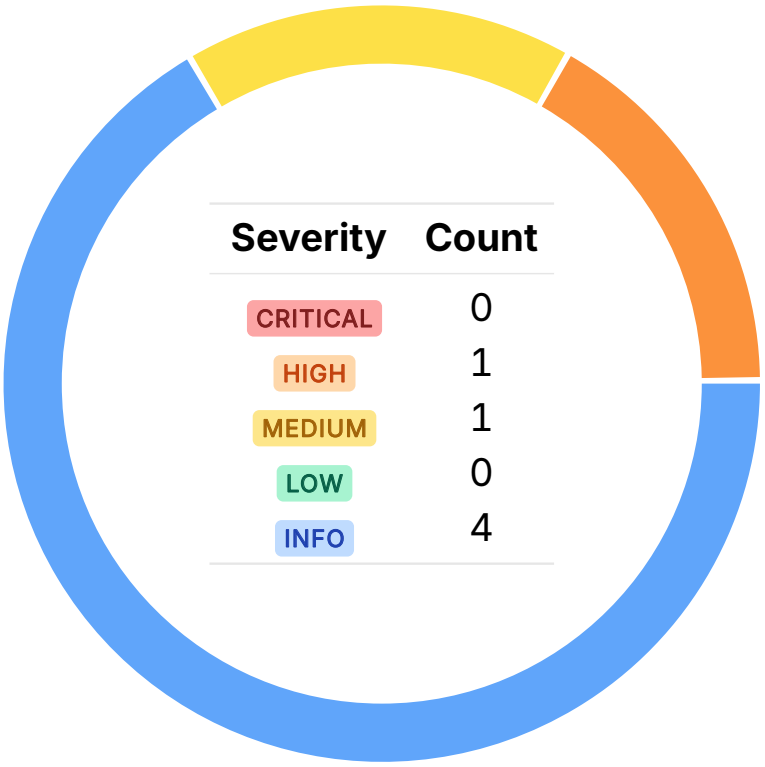
Name	Description
cetus-dlmm	A bin-based concentrated liquidity AMM on Sui that enables users to provide liquidity in discrete price ranges and enables them to execute swaps.
integer-mate	A library of Move modules that provides signed integer and some integer math functions.
move-stl	A Move library providing core data structures such as skip list, option, and linked table, along with random number generation utilities.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 6 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	1
MEDIUM	1
LOW	0
INFO	4

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-CDM-ADV-00	HIGH	RESOLVED	The current implementation allows rewards to start before `REWARD_PERIOD_START_AT`, breaking the reward settlement calculations, resulting in pool operations failing and creating a denial-of-service scenario.
OS-CDM-ADV-01	MEDIUM	RESOLVED	Due to rounding in `mul_div_floor`, small deposits will yield a `delta_liquidity_share` of zero, resulting in users losing tokens without receiving any corresponding liquidity shares.

© 2025 Otter Audits LLC. All Rights Reserved.

Absence of Verification for Reward Start Timestamp HIGH OS-CDM-ADV-00

Description

In Cetus DLMM, rewards are expected to start on or after the `REWARD_PERIOD_START_AT` timestamp, which anchors all reward periods. Currently, `pool::add_reward` does not enforce this, allowing a reward to be scheduled with a start time before this timestamp. When such a reward exists, `reward_settle`, which is called on every pool operation, will encounter invalid time intervals, breaking settlement calculations as `reward_settle` logic expects rewards to start on or after `REWARD_PERIOD_START_AT`. This results in a denial-of-service affecting multiple operations.

Remediation

Enforce `start_time >= REWARD_PERIOD_START_AT` during reward addition in `pool::add_reward` and `reward::add_reward`.

Patch

Resolved in cc0ed19.

© 2025 Otter Audits LLC. All Rights Reserved.

Improper Rounding MEDIUM OS-CDM-ADV-01

Description

The calculation of `delta_liquidity_share` in `bin::increase_liquidity` utilizes floor division, which may round down small values to zero. This implies that users depositing small token amounts may not receive any liquidity shares, effectively losing their deposited funds.

>_ *packages/cetus_dlmm/sources/bin.move* RUST

```
public(package) fun increase_liquidity([...]): (u64, u64, u128) {
    [...]
    let delta_liquidity_share = if (bin.liquidity_share ==0) {
        delta_liquidity
    } else {
        full_math_u128::mul_div_floor(
            bin.liquidity_share,
            delta_liquidity,
            bin_liquidity,
        )
    };
    [...]
}
```

Remediation

Add an assertion to ensure `delta_liquidity_share > 0` to prevent such rounding-related losses.

Patch

Resolved in 6dd44b8.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-CDM-SUG-00	`create_partner` allows the `PartnerCap` to be sent to `0x0`, which will result in locking the partner's referral fees permanently.
OS-CDM-SUG-01	The fee and reward accumulation logic may overflow due to unchecked additions, potentially corrupting balances.
OS-CDM-SUG-02	Recommendation for modifying the codebase to improve functionality and remove unnecessary code.
OS-CDM-SUG-03	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

© 2025 Otter Audits LLC. All Rights Reserved.

Invalid Recipient Address OS-CDM-SUG-00

Description

`partner::create_partner` currently does not validate the recipient address, allowing it to be set to `0x0`. Sending the `PartnerCap` to the zero address will render the capability unclaimable, permanently locking the partner's accumulated referral fees.

Remediation

Add a check to ensure `recipient != 0x0` in `create_partner`.

Patch

Resolved in cc0ed19.

© 2025 Otter Audits LLC. All Rights Reserved.

Risk of Overflow in Fee/Reward Calculation OS-CDM-SUG-01

Description

`update_fee_per_bin` and `update_reward_per_bin` add new fee or reward amounts to accumulated totals utilizing normal addition, which may overflow if the values become very large. Since these values are derived from wrapping arithmetic, an overflow will silently wrap around, affecting the accounting state and potentially resulting in users losing fees or rewards.

Remediation

Add explicit assertions for overflows or utilize saturating addition when increasing the fee/reward amounts.

Patch

Resolved in 6dd44b8.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-CDM-SUG-02

Description

1. `price_math::bin_bound` unnecessarily returns a constant via a runtime call, resulting in overhead. Replace it with a `macro` to allow compile-time inlining of `BIN_BOUND` which saves gas.

   >_ *packages/cetus_dlmm/sources/price_math.move* RUST

   ```
   /// Gets the bin boundary value.
   ///
   /// This function returns the bin boundary constant (443,636) which defines
   /// the maximum absolute value for bin IDs.
   ///
   /// ## Returns
   /// - `u32`: The bin boundary value (BIN_BOUND)
   public fun bin_bound(): u32 {
       BIN_BOUND
   }
   ```

2. `registry::create_pool` declares `config` as `&mut GlobalConfig` but never modifies it, rendering the `mut` unnecessary. Remove the mutable keyword.
3. The current `RewardConfig` and `BinStepConfig` structures have `copy` and `drop` capabilities, allowing users to store independent copies of the configuration. This creates a risk where users may later utilize a removed configuration to create pools or claim rewards, bypassing admin control. Remove `copy` and `drop` to return references instead of copies in functions, which restricts users to only reading the live configuration without storing it.

Remediation

Incorporate the above refactors.

Patch

1. Issue #1 was resolved in 05a78aa.
2. Issue #2 was acknowledged by Cetus.
3. Issue #3 was acknowledged by Cetus.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity

OS-CDM-SUG-03

Description

1. Replace the repetitive getter functions for constants with macros in `constants`, to reduce boilerplate and improve efficiency.
2. `skip_list::new` does not currently check if `max_level > 0`. Validate that the skip list always has at least one level to ensure correctness.
3. `bin::resolve_bin_position` hardcodes 16 bins per group utilizing bitwise operations (`>> 4` and `& 0xF`). This renders it inflexible if the number of bins per group changes. It will be appropriate to replace the hardcoded values with a constant such as `constants::MAX_BIN_PER_GROUP`, improving maintainability in case the number of bins per group needs to be changed.

   >_ packages/cetus_dlmm/sources/bin.move RUST

   ```rust
   /// Resolves the bin position from a score.
   ///
   /// ## Parameters
   /// - `score`: The score to resolve
   ///
   /// ## Returns
   /// - `(u64, u8)`: Tuple of (group index, offset in group)
   public fun resolve_bin_position(score: u64): (u64, u8) {
       let group_index = score >> 4;
       let offset_in_group = (score & 0xF) as u8;
       (group_index, offset_in_group)
   }
   ```

4. Currently, `Q64x64` fixed-point calculations utilize raw `u128`, which has no distinction from regular `u128` values. This creates a risk where a `u128`, which is not meant to be in the non-`Q64x64` format, may be mistakenly utilized, resulting in silent calculation errors. To prevent this, utilize the standard Sui `uq64_64` type or define a custom wrapper type for `Q64x64` values.

Remediation

Implement the above-mentioned suggestions.

© 2025 Otter Audits LLC. All Rights Reserved.

Patch

1. Issue #1 was resolved in cc0ed19.
2. Issue #2 was resolved in 94e2acc.
3. Issue #3 was acknowledged by Cetus.
4. Issue #4 was acknowledged by Cetus.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.